Pretraining

Lecture 2: data, training and scale

Aarne Talman

29 October 2026

LDA-T318 | LLMs and Generative AI for NLP | University of Helsinki

Course overview

Wk	Dates	Tuesday	Thursday	Teacher
1	**27 / 29 Oct**	**Language models**	**Pretraining**	**Aarne**
2	3 / 5 Nov	Post-training	Lab 2: tune vs prompt	Aarne
3	10 / 12 Nov	Inference	Lab 3: serve	Aarne
4	17 / 19 Nov	Evaluation	Lab 4: evaluate	Jussi
5	24 / 26 Nov	RAG	Lab 5: RAG	Dmitry
6	1 / 3 Dec	Agents	Lab 6: agent	Dmitry
7	8 / 10 Dec	Course recap	Special topics (TBC)	All

Reusing a task and model across labs is recommended, not required.

There are no intermediate submissions; the final report and code archive determine the grade.

Lecture overview

1. The training loop *10 min*
2. Data *20 min*
3. Scaling laws *15 min*
4. Training at scale *15 min*
5. Mid-training *10 min*
6. What pretraining buys *10 min*
7. Lab 1 briefing *8 min*

Take away

By the end of today you should be able to explain how a language model is pretrained, estimate the compute and memory needed for a run, and identify the main choices in its data recipe.

The training loop

Pretraining uses the same next-token objective

$$\mathcal{L}(\theta) \;=\; -\frac{1}{T}\sum_{t=1}^{T} \log p_\theta(x_t \mid x_{<t})$$

In words: walk through the text one token at a time, ask how much probability the model gave to the token that actually came next, and average the penalty for being wrong. Same cross-entropy as Tuesday, now over a few trillion tokens.

Teacher forcing. In training the model always conditions on the *true* prefix, never on its own predictions. Every position is a supervised example, and the causal mask makes all of them computable in one forward pass.

So a sequence of 4096 tokens provides a prediction target at almost every position, all computed in parallel. **Why this matters:** unlike an RNN, a transformer can process all training positions in parallel. Generation remains sequential; we return to that difference in week 3.

Batches are measured in tokens

In LLM training, batch size is usually reported in **tokens**. The useful quantity is how many token positions contribute to one optimiser step.

Run	Tokens / step	Note
GPT-3 175B (2020)	3.2M	fixed through training
Llama 3 405B (2024)	4M → 8M → 16M	ramped, along with sequence length 4k → 8k
Your Lab 1 run	0.1M	one GPU, no accumulation needed

Gradient accumulation buys you a large batch on small hardware: split the batch into several micro-batches, add up their gradients, and take one step. With the loss scaled correctly, this approximates one larger batch, subject to numerical and stochastic differences.

Batch ramping starts small and grows. Early on the gradients are large, so a noisy estimate is good enough and a small batch gets more out of each token. Later the model needs the hardware efficiency of a big one.

Source: Brown et al. 2020; Grattafiori et al., *The Llama 3 Herd of Models*, 2024.

Why AdamW is commonly used

Plain gradient descent takes the same size of step for every weight. AdamW keeps **two running averages per weight** and uses them to size each weight's step individually.

Momentum — the average of this weight's recent gradients. Smooths out the noise from one batch to the next.

Second moment — the average of its recent gradients *squared:* how big this weight's gradients typically are.

$$\text{step} = \text{learning rate} \times \frac{\text{smoothed gradient}}{\text{typical gradient size}}$$

In words: divide each gradient by how large that weight's gradients usually are, so every weight moves by a comparable *relative* amount whatever scale it lives on.

What it costs

Those two averages are two extra full-size copies of the model, held in fp32.

That is **8 bytes per parameter** of optimiser state, about half of the total training memory. It is the main reason a model you can serve on one card still will not train on it.

The **W** is decoupled weight decay: shrink every weight a little towards zero on each step, applied to the weights directly rather than pushed through the gradient.

Source: Loshchilov & Hutter 2019; hyperparameters as used in GPT-3, Llama and most open 2024–2026 recipes.

Keeping a training run stable

Gradient clipping. If the gradient as a whole is longer than a fixed threshold, almost always 1.0, scale it back down to that length. One bad batch then costs you a single bounded step rather than the run.

Warmup. AdamW's running averages are poor estimates for the first few hundred steps, so start the learning rate near zero and ramp it up over roughly 0.1–2% of training.

QK-Norm. From Tuesday: normalise the queries and keys inside attention. Adopted mainly because it suppresses loss spikes.

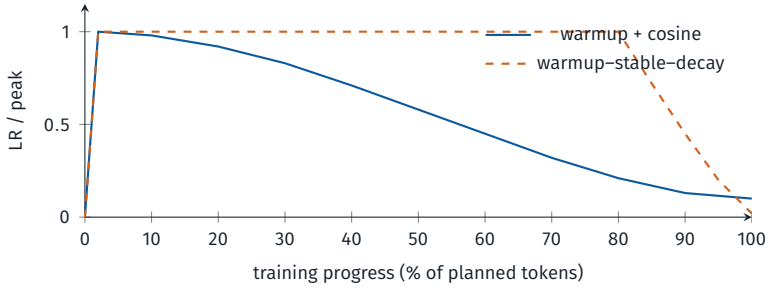
z-loss

A small extra penalty that keeps the model's raw output scores from drifting to ever-larger absolute values. That drift can build slowly and become expensive to correct late in a run. PaLM introduced this technique, and some later large-model recipes use it.

None of these make the model better. They are there so that the run reaches the end.

Source: Chowdhery et al., *PaLM*, 2022; Raschka, *A Dream of Spring for Open-Weight LLMs*, Feb 2026, on QK-Norm.

Learning-rate schedules: cosine versus WSD

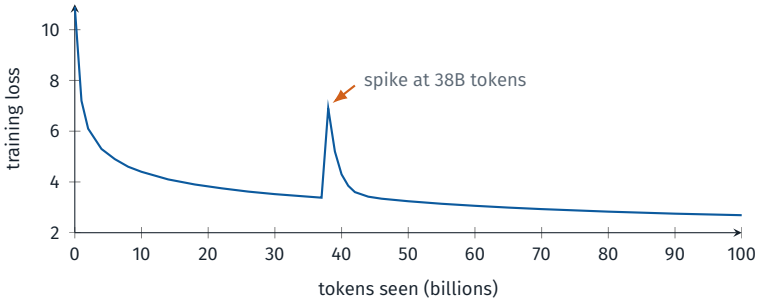


Cosine needs the total token budget declared *in advance*. Stop early and you stop at a high learning rate, with a model that is measurably worse than its loss curve suggests it should be.

WSD holds the peak rate flat, then decays sharply at the end. You can branch from *any* point in the stable phase, anneal on a different mixture, and get a finished model.

Source: Hu et al., *MiniCPM*, 2024; used in OLMo 2, SmolLM3 and several recent open-model recipes.

Loss spikes



Causes: a corrupted or pathological data shard; learning rate too high for the current curvature; fp16 overflow; router collapse in an MoE; a silently faulty GPU.

Mitigations: QK-Norm and z-loss up front; skip the offending batches; roll back to a checkpoint about 100 steps earlier and skip 200–500 batches of data. PaLM did exactly that, about twenty times.

Source: Chowdhery et al., *PaLM*, 2022.

A HEALTHY CURVE, AND A COMPARISON TO AVOID

What good looks like in Lab 1

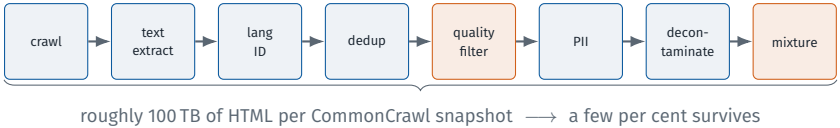
- a near-vertical drop in the first few hundred steps, as the model learns the unigram distribution
- a long, smooth, slowly flattening descent on a log-x axis
- validation tracking training, with a gap that does not widen
- no early flat plateau; if the loss stops improving almost immediately, check the learning rate and data loader

A lower loss is not always a better model

Cross-entropy is measured **per token**, so it is denominated in whatever your tokenizer calls a token. A larger vocabulary gives you fewer, harder tokens per sentence, so an equally good model records a *higher* loss. Put more repetition in the mixture and the loss drops even though the model is worse. Loss only compares within one tokenizer and one data mixture. Across them, use **bits per byte**, or downstream accuracy.

Data

The pipeline



Each stage removes unwanted material, but it can also remove useful text. We will look at what each stage is for, where it can fail, and why the trade-offs differ across languages.

Take away

The data pipeline strongly affects what a model learns, yet model reports often document it less fully than the architecture.

Stage 1: the crawl

CommonCrawl is a non-profit that has crawled the web roughly monthly since 2008. Each snapshot is a few billion pages.

It ships two formats, and the choice between them is the first quality decision in the pipeline:

- **WARC** — the raw HTTP response. Full HTML, headers, the lot.
- **WET** — CommonCrawl's own plain-text extraction, already done for you.

WET is convenient, but its extraction often leaves navigation bars, cookie banners, footers and link farms in the text.

Failure mode of this stage

You inherit somebody else's definition of what a web page says. Every serious 2024–2026 dataset — FineWeb, RefinedWeb, Dolma, HPLT — starts from **WARC** and does its own extraction. The crawl is not neutral either. It respects `robots.txt`, and it under-samples paywalled and login-gated text while over-sampling whatever is cheap to serve.

Source: CommonCrawl; Penedo et al., *The FineWeb Datasets*, 2024; Penedo et al., *RefinedWeb*, 2023.

Stage 2: text extraction

Turning HTML into the article, and only the article.

What must go: navigation, sidebars, cookie notices, “related articles”, comment spam, licence footers, JavaScript.

What must stay: the body text, in reading order, with paragraph and list structure intact.

trafilatura is the standard tool. FineWeb measured the swap directly: WET $\rightarrow$ trafilatura on WARC improved downstream benchmark scores on models trained on the output, at the cost of a slower and much more expensive extraction step.

Failure mode of this stage

Extraction errors come in **systematic** patterns rather than at random. A template that strips the first paragraph of every page on a large CMS removes exactly the lead sentences, the densest text on the page, across millions of documents at once. You will not notice this by sampling, because each individual document still looks fine.

Source: Penedo et al., *The FineWeb Datasets*, 2024; Barbaresi, *trafilatura*, 2021.

Stage 3: language identification

A classifier, usually fastText trained on Wikipedia and web text, gives each document a language and a confidence. You keep what clears a threshold.

Where it breaks	Consequence
short documents	unreliable under about 20 words, so short texts vanish
code-switching	a Finnish post quoting English clears neither threshold
related languages	Danish/Bokmål and Czech/Slovak get confused
the threshold itself	FineWeb keeps English above a confidence of 0.65

Each threshold trades quality against quantity, and a smaller language cannot give up quantity.

Source: Joulin et al., fastText, 2016; Penedo et al., *The FineWeb Datasets*, 2024.

Stage 4: deduplication

Three levels, usually all three:

- **exact** — hash the document, drop repeats. Cheap, catches mirrors and reposts.
- **near-duplicate** — hash overlapping word sequences and compare the hashes (MinHash and LSH), dropping pairs that share about 80% of them. Catches templated and lightly edited pages.
- **line-level** — drop lines repeated across many documents. This is what removes the surviving boilerplate.

The FineWeb surprise

Deduplicating *globally* across all 96 CommonCrawl snapshots gave **worse** models than deduplicating within each snapshot. Why: text that survives on the web across many years and many sites is disproportionately *good*. Aggressive global dedup removes it and leaves a residue of unique, never-copied, low-quality pages. So more filtering does not always give you a better corpus.

Why it helps: duplicated text is effectively upweighted, which inflates memorisation and leaks test sets into training.

Source: Lee et al., *Deduplicating Training Data Makes LMs Better*, 2022; Penedo et al., *The FineWeb Datasets*, 2024.

Stage 5a: quality filtering by hand

Hand-written heuristics, inherited largely from C4 and Gopher. A document survives if, roughly:

- 50–100 000 words, mean word length 3–10 characters
- fewer than 30% of lines ending in an ellipsis
- at least 80% of words containing an alphabetic character
- a sentence-ending punctuation mark on most lines
- symbol-to-word ratio below 0.1
- no match against a list of blocked terms

What these rules really are

A set of cheap proxies for "written by a person, for a reader". They are also **English-shaped**. A mean word length of 3–10 characters is wrong for Finnish or German compounding, and sentence-final punctuation is wrong for Thai. Blocked-term lists remove sex education along with pornography, and they take out a lot of text written by and about minority groups.

The rules are cheap and you can audit them. They also put a ceiling on quality, because a rule you can write down is a rule a spammer can satisfy.

Source: Raffel et al., *C4*, 2020; Rae et al., *Gopher*, 2021; Dodge et al., *Documenting the English C4*, 2021.

Stage 5b: quality filtering by classifier

Replace the rules with a model. **FineWeb-Edu** is the canonical worked example.

1. Ask a large instruct model to rate 460k FineWeb documents 0–5 for *educational value*
2. Train a small, cheap classifier on those labels, on top of an off-the-shelf embedding model
3. Run it over all 15T tokens of FineWeb
4. Keep the documents scoring 3 or above

Result: **1.3T tokens** out of 15T, so under a tenth survives. At the same token budget, models trained on the survivors are substantially better on knowledge-and-reasoning benchmarks such as MMLU and ARC.

The cost of the trick

You have encoded one model's taste into your corpus. "Educational value" favours textbooks, tutorials and encyclopaedic prose, and it disfavours dialogue, fiction, argument and informal registers. If you want a model that can write, that is the wrong thing to filter on, and you will discover it after the run.

Source: Lozhkov et al., *FineWeb-Edu*, 2024; Penedo et al., *The FineWeb Datasets*, 2024.

How to compare data pipelines

Inspection is necessary, but not sufficient

Samples and corpus statistics can reveal obvious problems. To determine which pipeline produces better training data, train comparable small models and evaluate them on the same tasks.

The FineWeb ablation protocol

- fix the architecture, tokenizer, hyperparameters and token budget
- train a 1.8B-parameter model on 28B tokens from each candidate dataset
- evaluate on a fixed suite of benchmarks chosen in advance for low variance and a signal above chance at that scale
- change **one** pipeline decision at a time

Source: Penedo et al., *The FineWeb Datasets*, 2024.

Why this is the methodological lesson

It turns data engineering into something you can run experiments on. It is also expensive: every filter you want to justify costs a training run, which is why the published ablation studies come from the handful of teams willing to spend compute on questions rather than on models. The same protocol is what Lab 1 Part C asks you to run, in miniature.

Stage 7: decontamination

Remove from the training corpus any text that overlaps the *test sets* you intend to report.

How: *n*-gram overlap against every benchmark's test split. GPT-3 used 13-grams; later work uses 8- to 13-grams or normalised substring matching. Flagged documents are dropped, or the matching passage is excised.

Why it is hard

- you must know your benchmarks before you train
- paraphrases and translations are invisible to *n*-gram matching
- a later benchmark can still overlap with earlier source material, paraphrases, translations or later training stages

Why you should care in week 4

Contamination can inflate a benchmark score without an obvious warning in the output. It is one possible explanation for an unexpectedly large result, and one reason Jussi's evaluation lecture emphasises held-out data you control. In your own project: your held-out set must never have been used during development, for exactly the same reason at a smaller scale.

Source: Brown et al. 2020; Soldaini et al., *Dolma*, 2024; Raschka, *Understanding the 4 Main Approaches to LLM Evaluation*, 2025.

Stage 8: the mixture

The last decision is how much of each source goes in, and how many times each one is seen. It has more effect on the model than anything before it.

Source	Typical share	What it contributes
filtered web	60–85%	breadth, world knowledge, register variety
code	5–20%	reasoning in general, not only coding
maths	2–10%	arithmetic and multi-step derivation
books, papers	2–10%	long-range coherence, formal register
curated reference	1–5%	tiny, and upsampled aggressively

Upsampling means sampling a scarce source more often. One study found useful gains over several passes and diminishing returns with heavy repetition; the useful limit depends on the corpus and model.

Source: Muennighoff et al., *Scaling Data-Constrained Language Models*, 2023; Aryabumi et al., *To Code or Not To Code?*, 2024.

A SMALLER LANGUAGE HAS A SMALLER DATA BUDGET

- English is roughly **45%** of CommonCrawl documents; Finnish roughly **0.2%**
- after filtering, usable Finnish web text is on the order of **tens of billions** of tokens

A compute-optimal 7B model uses about 140B tokens, and many recent recipes use substantially more. Finnish data alone is therefore not enough for this kind of general-purpose model.

The consequence

A practical approach is a **multilingual** model in which Finnish is upsampled. This supplies enough varied training text while giving Finnish a larger share than its raw web frequency. It can also transfer useful patterns learned from English and code to Finnish contexts.

Source: CommonCrawl language statistics; TurkuNLP Finnish web corpora; Luukkonen et al., *Poro 34B*, 2024.

Ways to increase training data for a smaller language

Approach	What it provides	Main limitation
multilingual mixture	more total data and cross-lingual transfer	less compute spent directly on the target language
repeat scarce data	greater weight for existing target-language text	diminishing returns and more memorisation
machine translation	large volumes from high-resource languages	translation patterns and MT errors
synthetic text	controllable topics and formats	generator bias, errors and licence constraints
parallel text	explicit links between languages	limited volume and domain coverage

These approaches are usually combined. Poro, for example, mixed Finnish with English and code, repeated Finnish data, and included aligned English–Finnish text.

Source: Muennighoff et al. 2023; Luukkonen et al., *Poro 34B*, 2024.

Case study: Poro and Viking on LUMI

Poro 34B (2024) — Silo AI with TurkuNLP and HPLT, trained on LUMI on roughly **1T tokens**:

- English (SlimPajama), the majority of the mixture
- *all* the usable Finnish there was — on the order of 30B tokens, repeated several epochs
- a large code share (StarCoder)
- explicit **English–Finnish translation pairs**

Viking 7B / 13B / 33B — the follow-up: about 2T tokens across Finnish, English, Swedish, Danish, Norwegian, Icelandic and code.

What it bought

Finnish performance far beyond anything trainable on Finnish alone, and English did not suffer for it. That is the cross-lingual transfer argument, demonstrated at scale on a national supercomputer.

What I would do differently

The architecture was frozen roughly nine months before release, and the field moved. A token budget that looked ambitious at kick-off was unremarkable at launch. And a large share of the engineering effort went into the software stack rather than the model.

Source: Luukkonen et al., *Poro 34B and the Blessing of Multilinguality*, 2024; Viking model cards, Silo AI / TurkuNLP / HPLT, 2024. Mixture shares approximate.

Licensing, and the open-data question

What is actually released

- **weights only** — almost everything called “open”: Llama, Qwen, Gemma, DeepSeek, Mistral
- **weights + data + code + recipe** — OLMo, and very little else
- **data only** — FineWeb, Dolma, HPLT, The Stack

The legal status of training on copyrighted web text is unsettled and differs between the EU and the US. The EU’s text-and-data-mining exception has an opt-out that publishers increasingly exercise.

Why this matters for research

If the data is secret, the experiment is not reproducible, and claims about architecture cannot be separated from claims about data. That is the concrete reason I said on Tuesday that “which architecture is best” is unanswerable from public evidence.

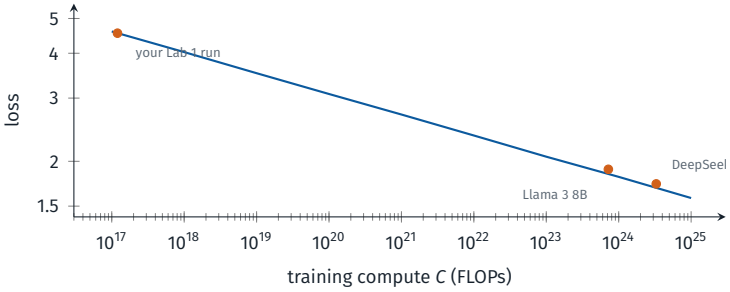
For your project: name the licence of every model and dataset you use, and check that it permits academic use. This is in the rubric.

Take away

At the same scale, what separates two models is mostly what they were trained on, and that is the part of a recipe least likely to be documented.

Scaling laws

Loss is a power law in compute



Kaplan et al. (2020): across seven orders of magnitude, test loss falls approximately as a **power law** in parameters, data and compute within the range they measured. On log–log axes this appears close to a straight line.

This is what made scaling an *engineering* activity. You can fit the line at small scale and predict the loss of a run you have not yet paid for. Frontier labs now do exactly that before committing.

Source: Kaplan et al., *Scaling Laws for Neural Language Models*, 2020. Curve is illustrative; the markers use $C = 6ND$ from the next slides.

Chinchilla: how to split a fixed budget

You have a fixed compute budget. Do you spend it on a **bigger model** (N parameters) or on **more text** (D tokens)?

Hoffmann et al. fitted the loss as a function of both, then asked which split minimises it with the budget $C = 6ND$ held fixed. The answer:

$$\frac{D}{N} \approx 20 \text{ tokens per parameter}$$

In words: in their experiments, model size and data size should grow **together**. The value 20 is a useful historical reference, not a universal constant.

The demonstration. Gopher: 280B parameters on 300B tokens. Chinchilla: **70B** parameters on **1.4T** tokens. Same training compute, but a smaller model trained on much more data. Chinchilla reported lower loss and stronger results on many benchmarks.

How to use the result. Treat the ratio as a starting point. Architecture, data quality, optimiser and learning-rate schedule all affect the fitted optimum, so current projects run their own small-scale studies.

Source: Hoffmann et al., *Training Compute-Optimal Large Language Models*, 2022.

Where $6ND$ comes from

A **FLOP** is one floating-point operation: a single multiply or a single add. Using one weight on one token means one multiply and one add, so **2 FLOPs per weight per token**.

- **Forward pass.** Every token passes through every parameter once. $2ND$
- **Backward pass.** Twice the work of the forward pass, because each layer computes two gradients: one to pass further back down the network, one to update its own weights. $4ND$

$$\boxed{C \approx 6ND} \qquad N = \text{parameters}, \quad D = \text{training tokens}$$

In words: a training run costs about six operations per parameter per token for the parameterised layers of a conventional dense transformer. The next slide lists important costs this estimate omits.

What $6ND$ leaves out

Attention's quadratic term. Comparing every token with every other token uses *no parameters*, so $6ND$ cannot see it at all. At a 4k context window it adds about **10%** on top. The work grows with the square of the context length, so at 128k context it dominates the bill.

Embeddings. Usually left out of the parameter count. For a 70B model you can ignore them. For a 20M model with a 32k vocabulary they are a large fraction of the whole thing.

Activation checkpointing

Save fewer intermediate activations and recompute them during the backward pass. Full recomputation adds roughly one forward pass:

$$C \approx 8ND \quad \text{rather than} \quad 6ND$$

Selective checkpointing adds less compute. The exact trade-off depends on which operations are recomputed.

So $6ND$ is a *lower bound on the useful work*. Measured wall-clock FLOPs are always higher.

Worked: your Lab 1 run

Your model has **20 million** parameters and you train it on **1 billion** tokens, at six operations each:

$$C = 6ND = 6 \times (2 \times 10^7) \times (1 \times 10^9) = 1.2 \times 10^{17}\ \mathrm{FLOPs}$$

One GH200 does about 10^{15} of those operations per second at peak, and a good run turns perhaps 40% of peak into useful work. At 4×10^{14} per second you get through the whole budget in roughly **300 seconds**.

That is five minutes of pure arithmetic.

Measure this on Roihu

Because a 20M-parameter model **cannot keep an H100 busy**. The matmuls are too small, the data loader and the Python overhead dominate, and your real MFU will be a few per cent. Twenty minutes is a planning estimate, not a guarantee. Record the actual throughput and wall time; their gap from the arithmetic estimate is itself a useful result.

At cloud rates of roughly two euro per GPU-hour, your entire pretraining run costs well under one euro.

Source: GH200 peak bf16 throughput from NVIDIA datasheets; 40% MFU is a typical figure for a well-tuned large run, not for a tiny one.

Why deployed models are often trained past the Chinchilla optimum

Model	Params	Tokens	tokens / param
Chinchilla (2022)	70B	1.4T	20
Llama 3 8B (2024)	8B	15T	about 1 900
DeepSeek V3 (2024)	37B active	14.8T	about 400
Your Lab 1 run	20M	1B	50

Chinchilla-optimal is the wrong target for a model you will serve

Chinchilla minimises **training** compute for a given loss. If you are going to run this model billions of times, the quantity to minimise is **inference** compute, roughly two operations per parameter per generated token, which does not depend on how long you trained at all. So you overtrain deliberately: spend more now, to get the same loss from a permanently smaller model.

Llama 3 8B used roughly **90 times** the compute that Chinchilla would have prescribed for an 8B model, and Meta said explicitly that the loss was still improving when they stopped.

Source: Hoffmann et al. 2022; Grattafiori et al. 2024.

Loss is useful, but it is not the final goal

Scaling laws predict validation loss. Product and research decisions usually depend on something else: task accuracy, reliability, latency, memory use or cost per request.

Two models with similar loss can differ because of their data mixture, tokenizer, context length and post-training. A small loss improvement may also be too expensive to justify at inference time.

How scaling laws are used

Use small runs to forecast whether a larger pretraining run is healthy and what loss it may reach. Then evaluate the resulting model on the tasks and constraints that matter for its intended use. Loss is a training diagnostic and forecasting target; it does not replace downstream evaluation.

Take away

$C \approx 6ND$ estimates what a run costs. It does not tell us whether the resulting model is useful.

Training at scale

Why training uses much more memory than inference

What you must hold	Inference	Training
parameters, bf16	2 bytes	2 bytes
gradients, bf16	—	2 bytes
master parameters, fp32	—	4 bytes
Adam moments m and v, fp32	—	8 bytes
per parameter	**2 bytes**	**16 bytes**

Training also stores **activations**, which depend on batch size, sequence length and model depth.

Useful rule of thumb: 16 bytes per parameter for this mixed-precision AdamW setup. Exact requirements vary, but fitting for inference does not mean fitting for training.

Source: Standard mixed-precision AdamW accounting; cf. Rajbhandari et al., *ZeRO*, 2020.

Memory examples: 7B and 20M parameters

A 7B model, full fine-tune or pretrain

$$7 \times 10^9 \times 16 \text{ bytes} \approx 112 \text{ GB}$$

- one GH200 has **96 GiB**, about 103 GB
- so the *optimiser state alone* does not fit, before a single activation is stored
- you can serve this model on half a card, and training it needs rather more than one

Your Lab 1 model

$$2 \times 10^7 \times 16 \text{ bytes} \approx 320 \text{ MB}$$

- that is **0.3%** of the card
- you can use an enormous batch, and you should
- your bottleneck will be the data loader and kernel launch overhead, not memory

The entire rest of this section exists because of the left-hand box.

Three honest escape routes from the left-hand box: train something smaller; shard the state across many GPUs; or do not train all the parameters. We do the second one today and the third one next week.

Activations, and buying memory with compute

The backward pass needs the intermediate tensors the forward pass produced, the **activations**. Stored naively, the total is about

$$\text{batch} \times \text{sequence length} \times \text{model width} \times \text{layers}$$

In words: one number stored for every slot, in every layer, for every token in the batch.

Activation memory grows with batch size, sequence length, model width and depth. It becomes especially important when the context length increases.

Activation checkpointing

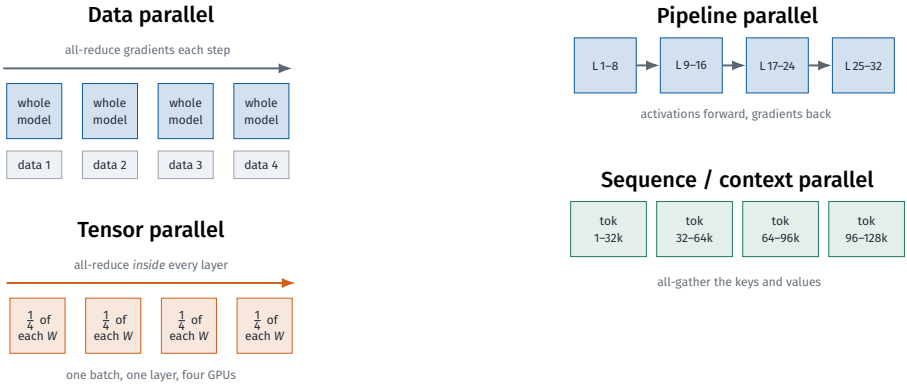
Store activations only at layer boundaries. Recompute the rest during the backward pass.

Memory: save selected layer boundaries rather than every intermediate tensor.

Compute: recompute the missing intermediates during backward. Full recomputation adds roughly one forward pass; selective methods add less. Selective variants recompute only what is cheap to recompute and expensive to store, which usually means attention rather than the feed-forward block.

Source: Chen et al., *Training Deep Nets with Sublinear Memory Cost*, 2016; Korthikanti et al., *Reducing Activation Recomputation*, 2022.

Four ways to split a training run



Two words will keep coming back. An **all-reduce** adds one tensor up across every GPU and hands the total back to all of them, and an **all-gather** gives every GPU a copy of every other GPU's piece. You pay for splitting in exactly that traffic. The four axes are **orthogonal**, and a large run uses three or four at once.

Data parallel, and ZeRO

Plain data parallel. Every GPU holds the whole model and processes a different slice of the batch; the gradients are then summed across all GPUs before the step.

- trivially simple, almost perfectly scalable in compute
- but every GPU still holds all 16 bytes per parameter

ZeRO / FSDP. To **shard** something is to give each GPU only its own slice of it and fetch the rest over the network when needed. ZeRO shards the two optimiser averages, then the gradients, then the weights themselves, collecting each layer's weights only for the moment that layer is in use.

Source: Rajbhandari et al., *ZeRO*, 2020; PyTorch FSDP, 2023.

Stage	What is split	GB/GPU
none	nothing	112
ZeRO-1	averages	38
ZeRO-2	+ gradients	26
ZeRO-3 / FSDP	+ weights	14

A 7B model across **8** GPUs. Only the last row fits on one card, and it pays for that by moving far more bytes over the network.

Precision

A number format splits its bits between the **exponent**, which sets how large or small a value it can represent, and the **mantissa**, which sets how many significant digits it keeps.

Format	Exponent	Mantissa	Why it matters
fp32	8	23	master weights and optimiser state
fp16	5	10	overflows; needs loss scaling
bf16	8	7	**fp32's range**, fewer digits
fp8 (E4M3)	4	3	2× throughput; per-block scaling

Why bf16 became common: it keeps fp32's exponent range, which reduces overflow and underflow problems compared with fp16, while retaining half-precision storage and matrix operations.

DeepSeek V3 used **fp8** for selected operations, with higher precision where needed. The potential speed-up depends on hardware, kernels and the scaling strategy.

Source: DeepSeek-AI, *DeepSeek-V3 Technical Report*, Dec 2024; Micikevicius et al., *Mixed Precision Training*, 2018.

MFU: HOW MUCH OF THE HARDWARE IS DOING MODEL WORK

Model FLOPs utilisation (MFU) compares estimated model arithmetic with the peak arithmetic the allocated GPUs could have performed:

$$\mathrm{MFU} = \frac{6ND}{\text{run time} \times \text{number of GPUs} \times \text{peak rate}}$$

In words: how much of the available compute became useful model operations?

Why it is below 100%

- GPUs wait for data and communication
- norms and softmax do not reach matrix-multiply peak
- checkpointing repeats work
- small matrices do not fill the GPU

What it is used for

Track MFU while tuning one training system. A sudden drop can reveal a data-loader, communication or kernel problem. Comparisons across models, hardware or numeric formats need care because both $6ND$ and "peak rate" are approximations.

Large training runs fail regularly

Llama 3 405B, over a **54-day** pretraining window on 16k H100s:

- **466** job interruptions
- **419** of them unexpected
- **78%** of the unexpected ones were hardware — GPU failures most of all
- effective training time still above **90%**, because the recovery was automated
- power draw across the datacentre swinging by tens of megawatts as thousands of GPUs ramped together

What this looks like from inside

Silent data corruption that shows up only as a loss that is slightly too high.

NaN hunts that take three days and end at one bad node.

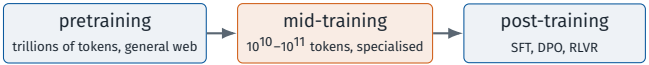
Checkpoint cadence as an explicit trade: a 405B optimiser state is about 6.5 TB, so writing it costs real minutes.

On LUMI, the same class of problems plus an immature software stack that absorbed a large share of the project's engineering.

Source: Grattafiori et al., *The Llama 3 Herd of Models*, 2024; personal experience, Poro / Viking on LUMI, 2023–2024.

Mid-training

Mid-training: targeted continued pretraining



Same objective, different data. Mid-training is still next-token prediction with AdamW. What changes is that the mixture is chosen deliberately rather than scraped.

Why it exists. The cheapest place to install a capability is usually the last few per cent of pretraining, rather than the start of a trillion-token run or a fine-tune on a hundred thousand examples.

The term only stabilised around 2024–2025. Before that the same work was called “annealing”, “the decay phase”, “continued pretraining” or nothing at all.

Source: OLMo 2 (Allen AI, 2024) is the clearest publicly documented example; Raschka, *The State of LLMs*, 2025, on the terminology.

Long-context extension: the problem

Pretraining at 128k context is wasteful: attention is quadratic, activation memory is linear in sequence length, and almost no documents are that long. So you pretrain short, at 4k or 8k, and extend afterwards.

Why you cannot just feed it more tokens. RoPE encodes position by rotating each attention dimension by an angle proportional to the token's position. Think of a bank of clocks, each ticking at a different speed, with the whole range set by a number called the **base**, usually 10^4.

Past the positions seen in training, the fast clocks have wrapped around many times into a combination of readings the model has never encountered. Attention degenerates.

Source: Su et al., RoFormer, 2021; Grattafiori et al. 2024.

Base scaling, the blunt fix

Increase the base: $10^4 \to 5 \times 10^5$ (Llama 3) or higher. Every frequency slows down, so the same position index maps to a smaller angle and more positions fit inside the range the model knows. It works, but you pay for it in resolution on *local* position, which is what the model leans on most.

Annealing on a higher-quality mixture

During the decay phase of a WSD schedule, swap the data mixture for a small, very high-quality one and let the learning rate fall to near zero.

Why use the decay phase? It provides a short, inexpensive stage in which the data mixture can be changed and compared. A useful intuition is that smaller late updates refine the model without repeating the full pretraining run; the exact mechanism is still an empirical question.

OLMo 2 is the documented example: a curated mixture of high-quality web, maths and instruction-like data, applied over the final 50B to 300B tokens, then several annealed runs averaged into one model.

Source: OLMo 2 (Allen AI, 2024); Hu et al., *MiniCPM*, 2024.

Why WSD and annealing fit together

A cosine schedule has no flat phase to branch from, so annealing means re-planning the whole run. With WSD you keep one stable-phase checkpoint and anneal it several times over, varying the mixture, the budget or the target language, for a few per cent of the cost each. This flexibility is an important reason WSD became common.

What targeted data can add

What actually goes into a mid-training mixture, and why:

Ingredient	What it is for
maths	symbolic step chains; the best lever on reasoning
code	structure and verifiable chains; lifts reasoning
long documents	teaches the model to *use* its context window
instruction-like text	question-answer formatting for SFT to grip
target language	the cheapest place to add a new language
synthetic textbooks	dense explanatory prose; narrows the distribution

By the bottom two rows, mid-training is hard to tell apart from SFT without the loss masking.

Source: OLMo 2 (2024); Aryabumi et al., *To Code or Not To Code?*, 2024; Gunasekar et al., *Textbooks Are All You Need*, 2023.

What pretraining buys

In-context learning, live

Demo: a base model, with no instruction tuning and no chat template.

```
Review: The food was cold and the waiter ignored us.  Sentiment: negative
Review: Beautiful room, spotless, and the staff were kind.  Sentiment: positive
Review: It arrived late and the box was crushed.  Sentiment: negative
Review: Exactly what I wanted, and cheaper than the shop.  Sentiment:
```

The model completes `positive`. Nobody trained it to classify sentiment, and no weight was updated.

It picked up sentiment classification **from the prompt**, because the prompt is a document whose most likely continuation is the correct label.

In-context learning is the emergent capability that made the whole prompting era possible. You get it free from next-token prediction on a corpus that happens to contain lists, tables and worked examples.

Source: Brown et al., *Language Models are Few-Shot Learners*, 2020.

What in-context learning means

What happens. The prefix changes the model's *conditional distribution*. No weights move, and the effect disappears as soon as the context does.

What to be careful about. Calling it learning is a stretch. There is a real literature analysing it as gradient descent inside the forward pass, but the case is suggestive rather than settled.

Empirically: performance rises with the number of examples, with the consistency of the *format*, and surprisingly weakly with whether the labels in the examples are actually correct.

Why it emerges at all

The pretraining corpus is full of documents in which a pattern is established and then continued: glossaries, tables, translation pairs, exercise-and-answer sets, code with consistent conventions. A model that models those documents well *must* learn to infer a rule from a prefix and apply it. There was never a separate objective for this.

In Lab 1 Part D you run the same few-shot prompt on your tiny model and on a larger base model. Treat this as a qualitative comparison: scale, architecture, tokenizer and training data all differ.

Source: Brown et al. 2020; Min et al., *Rethinking the Role of Demonstrations*, 2022.

Now ask it a question

Demo: the same base model, same session, no few-shot examples.

```
Prompt:  What is the capital of Finland?

Output:  What is the capital of Sweden?
         What is the capital of Norway?
         What is the largest city in Finland?
         What is the population of Helsinki?
         Answer the following questions in complete sentences.
         1. What is the capital of Finland? _____________
         2. What is ...
```

Nothing has necessarily gone wrong here. More questions are one plausible continuation because question lists and worksheets occur in the pretraining corpus.

And it will not stop. No token in the corpus reliably means “the assistant has finished its turn”, because there is no assistant.

What the base model does not have

Missing	Why pretraining could not supply it
stopping	no end-of-turn token exists in web documents
turn-taking	no notion of “user” and “assistant” roles
instruction following	instructions occur, but obeying them is rarely the likeliest continuation
a preferred answer	good and bad answers appear at their web frequencies
refusal	declining a request is rare, and inconsistent where it occurs
a consistent persona	the corpus is millions of different authors

Every row is a problem of **selection** rather than **knowledge**. The few-shot demo showed the base model can produce the right behaviour; it just does not reach for it by default. That is the thesis of next Tuesday.

Source: Ouyang et al., *InstructGPT*, 2022, for the original framing.

What pretraining gives us

What pretraining produced

- a compressed model of a large slice of human text
- world knowledge, syntax, code semantics, arithmetic, discourse structure
- the ability to infer a task from a prefix and continue it
- a representation good enough that a few thousand examples can redirect its behaviour

What it did not produce

- reliable assistant-style turn-taking, instruction following or stopping behaviour

Why base models are still released

A base model is a useful starting point when researchers want to study or shape behaviour without an existing instruction-tuning policy. Lab 2 compares a base model with a corresponding instruct model. You may use a different suitable pair from the one used elsewhere in your project.

Take away

Pretraining supplies broad language and task capability. Post-training can select, strengthen and sometimes add behaviours using much less compute, but it still depends on what the base model can represent.

Lab 1 briefing

Lab 1: from tokenizer to tiny GPT

A. Tokenizer *CPU Jupyter, about 1 h*

Byte-level BPE on a Finnish Wikipedia slice at two or three vocabulary sizes, then compare fertility against Qwen3.5 and Gemma 4 on FLORES-200.

B. Tiny GPT *GPU batch, about 20 min / run*

Train a 10–30M-parameter decoder with *your* tokenizer. Script and Slurm template provided.

C. Scaling *3 GPU batch jobs*

Three model sizes at a fixed token budget. Plot validation loss against compute, estimating compute as $6ND$.

D. Tiny versus pretrained *GPU Jupyter, about 20 min*

The same few-shot prompt on your tiny model and on a real base model.

E. Project *planning*

Choose a task, model and held-out set. You may revise this later.

How the parts connect to the lectures

the training loop and the loss curve $\Rightarrow$ B

$C \approx 6ND$ and scaling laws $\Rightarrow$ C

in-context learning $\Rightarrow$ D

and Tuesday's tokenization $\Rightarrow$ A

A complete run of the three supplied model sizes should use roughly **one GPU-hour** in total.

Start with `lab1-example/README.md`; it includes a CPU smoke test, the training script, configurations and plotting code.

How to check that a run is working

Check	Expected pattern	If it differs
first-step loss	close to $\ln(V)$; about 10.4 for 32k tokens	check initialization, vocabulary and labels
training loss	falls quickly, then more slowly	inspect the learning rate and batches
validation loss	follows training loss	check the split and possible overfitting
throughput	becomes stable after warmup	inspect the data loader and GPU utilisation
output files	metrics, summary and weights exist	read the end of the job log

The one-line sanity check: your initial loss should be close to the log of your vocabulary size, and $\ln 32000 \approx 10.4$. If your run starts far from that value, investigate before spending GPU time.

Material for the final report

1. a **fertility table** covering your tokenizers and two production tokenizers, across three languages
2. **loss curves** for all your runs, on one set of axes
3. a **scaling plot** of validation loss against $6ND$, log–log
4. one paragraph comparing **your run's compute** with a published frontier run, using $6ND$ for both
5. your project plan: task, model and held-out set

What makes the analysis useful

A number with the method that produced it, and a sentence saying what you concluded from it. If a result surprises you, investigate it and record what you checked. "The three runs did not order the way I expected, and here is what I checked" is a good paragraph.

Keep the scripts, configurations and machine-readable result files in your repository. There is no separate Lab 1 submission.

Working on Lab 1

- Roihu is down for maintenance on 3–4 November. Those are not available for GPU work.
- **There is no supervised lab this week.** The next time we are all in a room with GPUs is 5 November.
- Questions go to the course channel. It is watched all week and answers are visible to everybody.
- There is no intermediate submission. Keep your work for the final report and code archive.

If something breaks

Post the **job id**, the **command**, and the **last twenty lines** of the log. Three quarters of problems are solved from that alone. Do not post screenshots of a terminal.

A useful starting point

Run the bundled CPU smoke test, then Part A. This checks your environment and data flow before you request GPU time.

Three things to remember

1. $C \approx 6ND$ **gives a useful first estimate of training compute.** Two FLOPs per parameter per token forward and about four backward. Remember what the approximation leaves out.
2. **Data choices strongly affect model quality.** Inspect the data for obvious problems, then compare pipeline choices by training small, controlled models and evaluating them.
3. **A deployed model may be trained past the compute-optimal point.** Extra training can produce a smaller model with lower inference cost, trading a one-time expense for repeated savings.

And one number to carry: **16 bytes per parameter** to train in mixed precision, against 2 to serve.

Tuesday: the model that will not stop talking

We ended on a base model that answered a question with more questions and then kept going.

Nobody patches that. It is the gap between a model of *documents* and an *assistant*, and closing it takes a whole discipline:

SFT → preference optimisation → RLVR → distillation

Each stage exists because the previous one has a specific limitation.

Tuesday 3 November

Post-training. Chat templates, loss masking, DPO, GRPO, and why being able to *check* an answer removed the need for a reward model.

The base-versus-instruct demo will be pre-recorded, because Roihu is down that day.

This week: begin Lab 1 and keep the outputs for your final report. There is no intermediate submission.

Reading

Core

- Stanford CS336, *Language Modeling from Scratch*, the lectures on data and on systems
- Hugging Face, *The Smol Training Playbook*, 2025
- Karpathy, *nanoGPT*

Data

- Penedo et al., *The FineWeb Datasets*, 2024
- Lozhkov et al., *FineWeb-Edu*, 2024
- Soldaini et al., *Dolma*, 2024
- Luukkonen et al., *Poro 34B and the Blessing of Multilinguality*, 2024

Scaling

- Kaplan et al. 2020; Hoffmann et al. 2022 (Chinchilla)
- Muennighoff et al., *Scaling Data-Constrained Language Models*, 2023
- Schaeffer et al., *Are Emergent Abilities a Mirage?*, 2023

Systems

- Rajbhandari et al., *ZeRO*, 2020
- Shoeybi et al., *Megatron-LM*, 2019
- Grattafiori et al., *The Llama 3 Herd of Models*, 2024; read section 3 on infrastructure
- DeepSeek-AI, *DeepSeek-V3 Technical Report*, 2024, for fp8 training in detail

For current developments, Raschka's *Ahead of AI*: *A Dream of Spring for Open-Weight LLMs* (Feb 2026) and *The State of LLMs* (2025).